Transformers from scratch

1.1

Turn text into numbers, calculate its context, and choose what comes next.

How does ChatGPT work?

We will study the Transformer architecture.

The input is a sequence of tokens: small pieces of text.

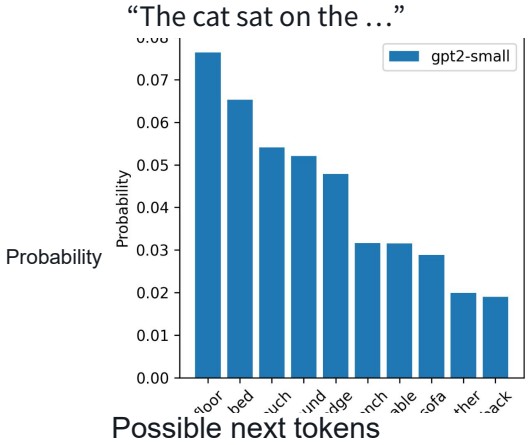
The output gives a probability for each possible next token.

Example model: small GPT2

Approximately 124 million parameters[1]

Approximately 40 GB of training text

[1] 32-bit weights alone: about 496 MB.
Running the model needs additional memory.



The output is a probability for each possible next token, rather than a completed sentence.



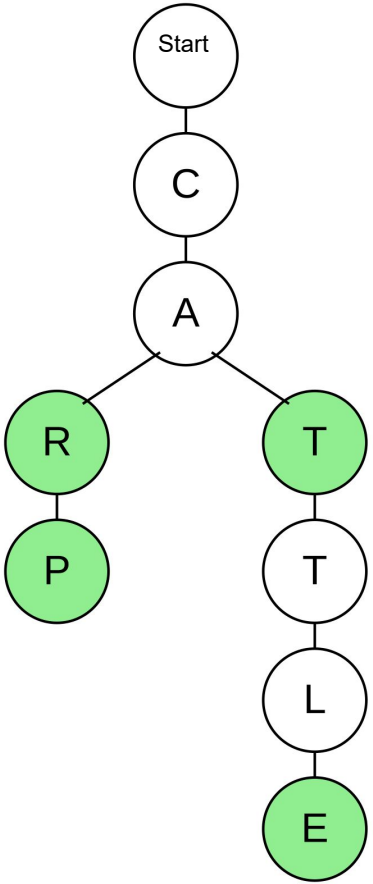
Enter your own sentence to explore how it is split into tokens.

Every token has an ID.

The model receives these IDs as input.

Further video: building a GPT tokenizer

A token is a piece of text; a token ID is the number assigned to that piece.



How does a tokenizer work?

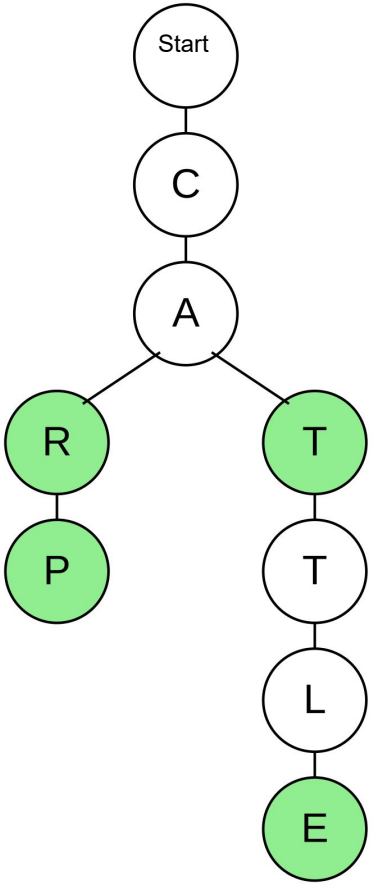
A vocabulary maps pieces of text to integers.

$$\{cat, cattle, car, carp\} \rightarrow \{0, 1, 2, 3\}$$

The tokenizer is prepared first and stays fixed during model training.

GPT2 uses byte pair encoding (BPE).

The vocabulary links text pieces to IDs. It is different from the table of learned vectors.



A closer look at tokenizers

A vocabulary maps pieces of text to integers.

$$\{cat, cattle, car, carp\} \rightarrow \{0, 1, 2, 3\}$$

A trie is a data structure for looking up strings.

Special tokens can mark document boundaries.

The diagram illustrates string lookup; distinguish it from GPT2's BPE.

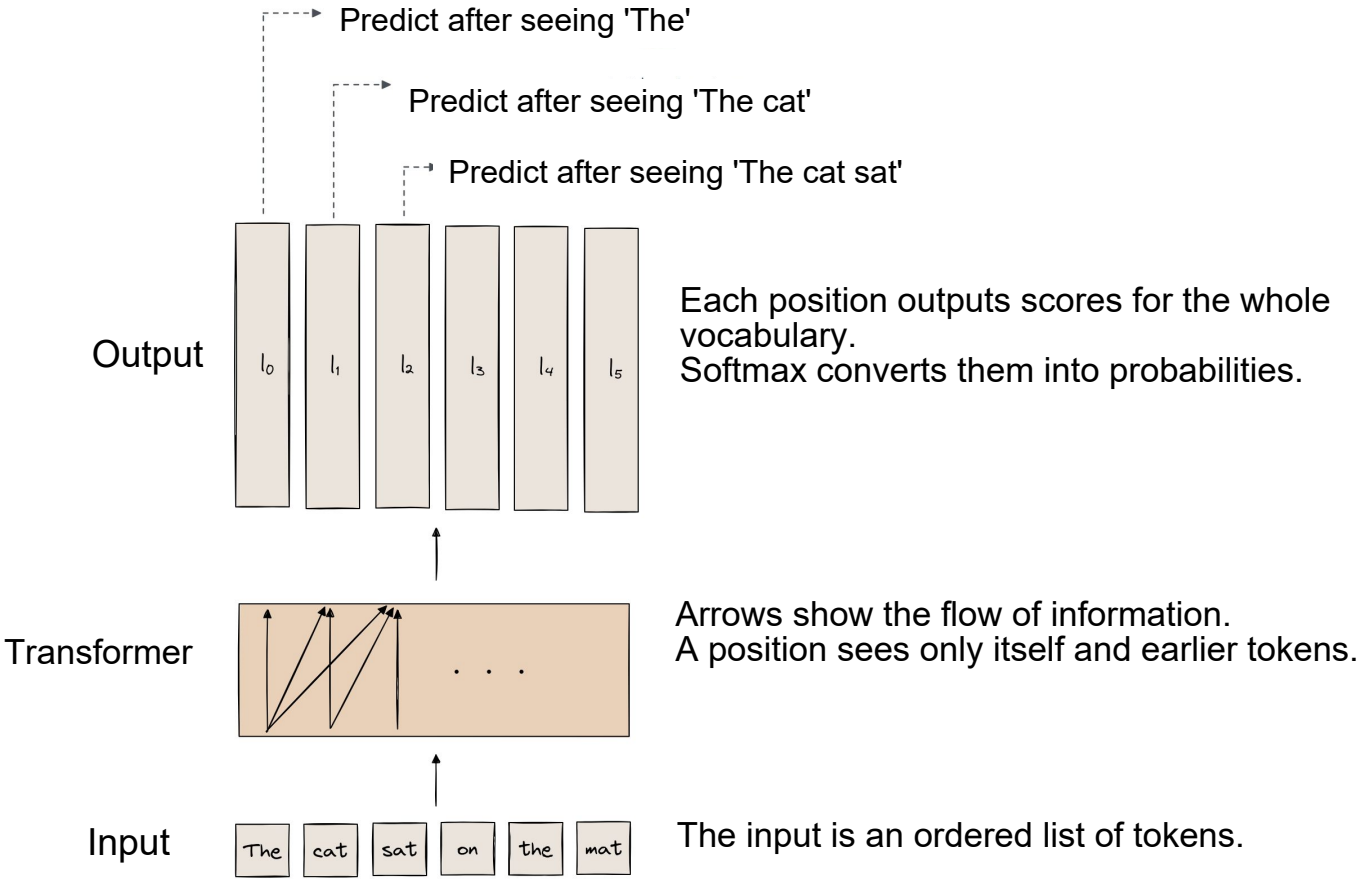
Today we use the existing GPT2 tokenizer.

Models can have different vocabularies and tokenization rules.

The tokenizer is prepared before model training and kept fixed.

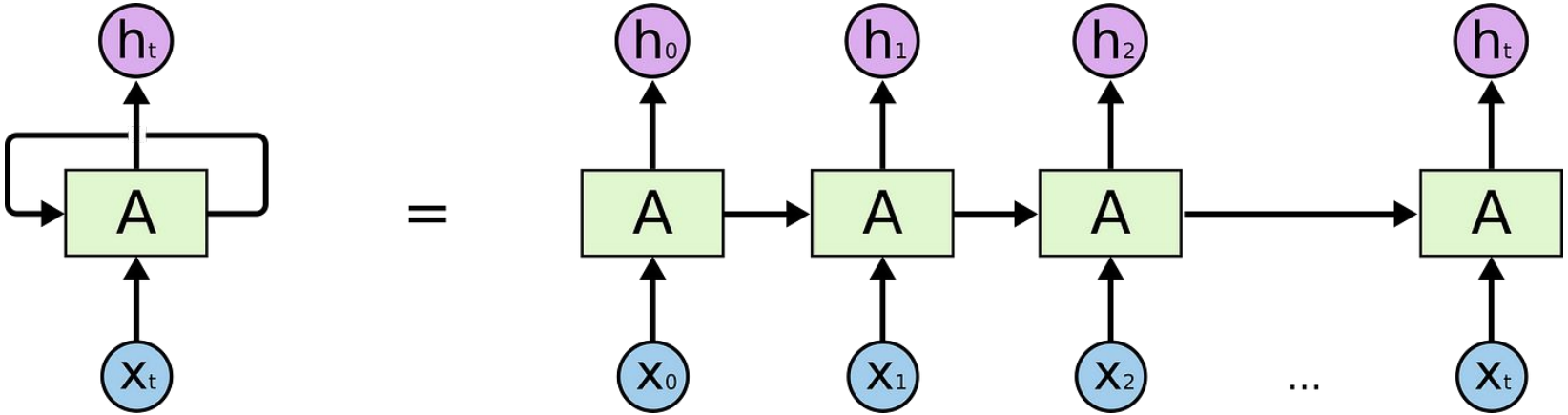
GPT2 uses byte pair encoding (BPE).

Today we use the existing GPT2 tokenizer and focus on what its inputs and outputs mean.



During training, predict the next tokens at several positions together while hiding future answers.

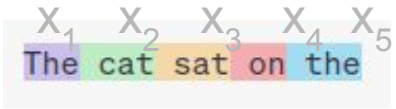
Within one sentence, an RNN waits for the preceding calculation.



An RNN waits for the previous step. A Transformer processes several positions together during training.

Generating text with a trained model: one token at a time

Start with an input sentence: 'The cat sat on the'



Pass its token IDs to the model to obtain probabilities for the next token.

Which token should we choose?

Choose the most probable token: this can cause repetition.

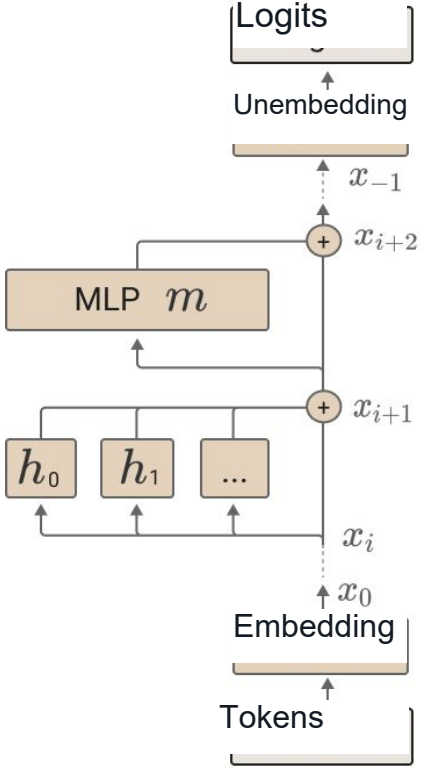
Sample by probability: higher-probability candidates are chosen more often.

Adjust temperature: raising it can make the choices more varied.

Append the selected token to the input.

Predict again with the longer input and repeat the selection process.

Predict, choose one token, and append it to the input. Repeat to produce a sentence.



Convert the final vector into scores for vocabulary items.

$$T(t) = W_U x_{-1}$$

Add the MLP's result to the existing information.

$$x_{i+2} = x_{i+1} + m(x_{i+1})$$

Add each attention head's result to the existing information.

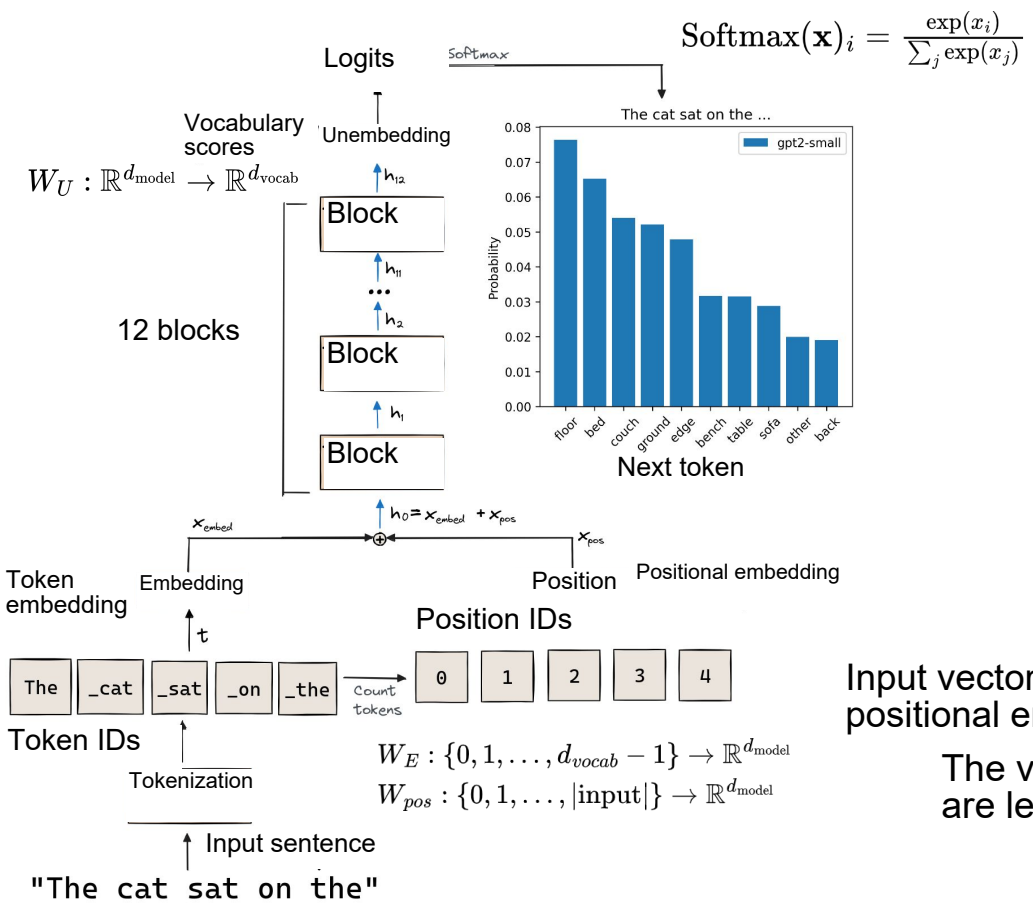
$$x_{i+1} = x_i + \sum_{h \in H_i} h(x_i)$$

One residual block

Turn tokens into vectors.

$$x_0 = W_E t$$

Keep the existing information and add the results of attention and the MLP in turn.

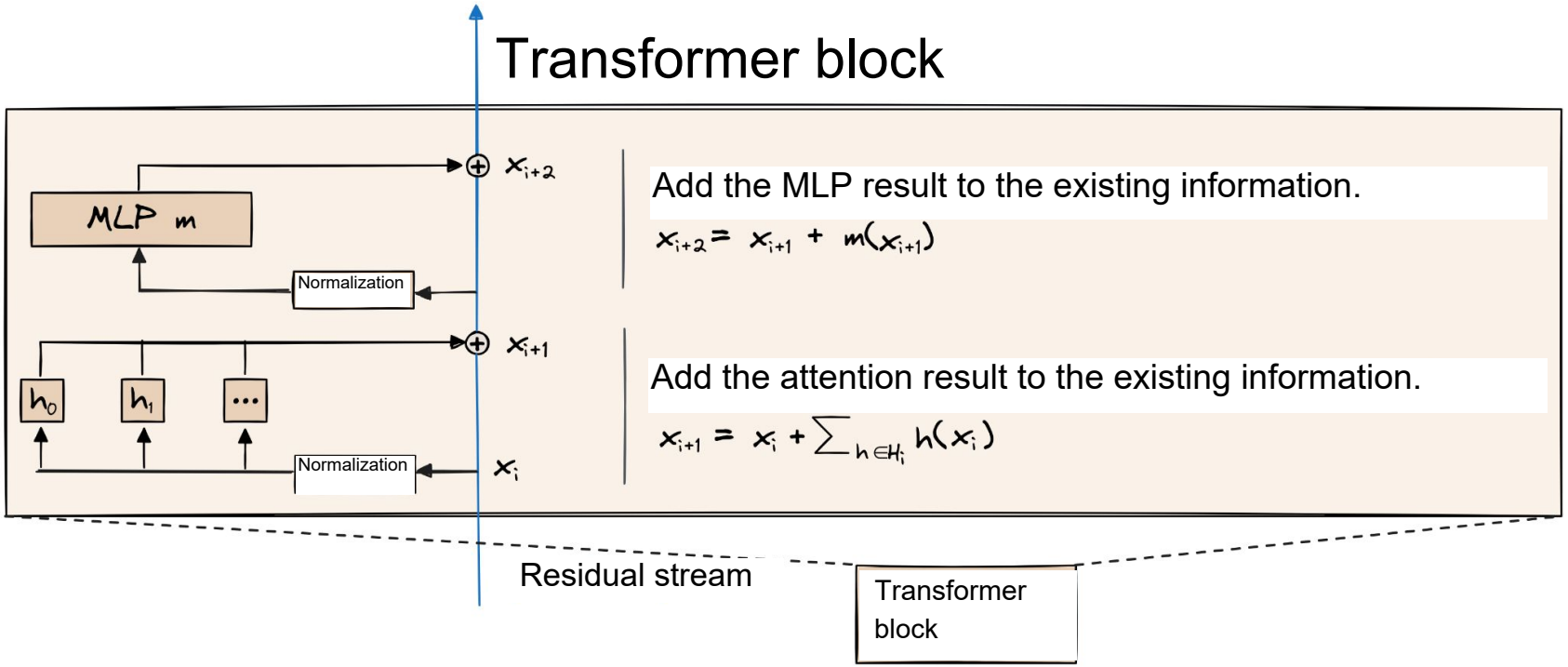


Input vector = token embedding + positional embedding

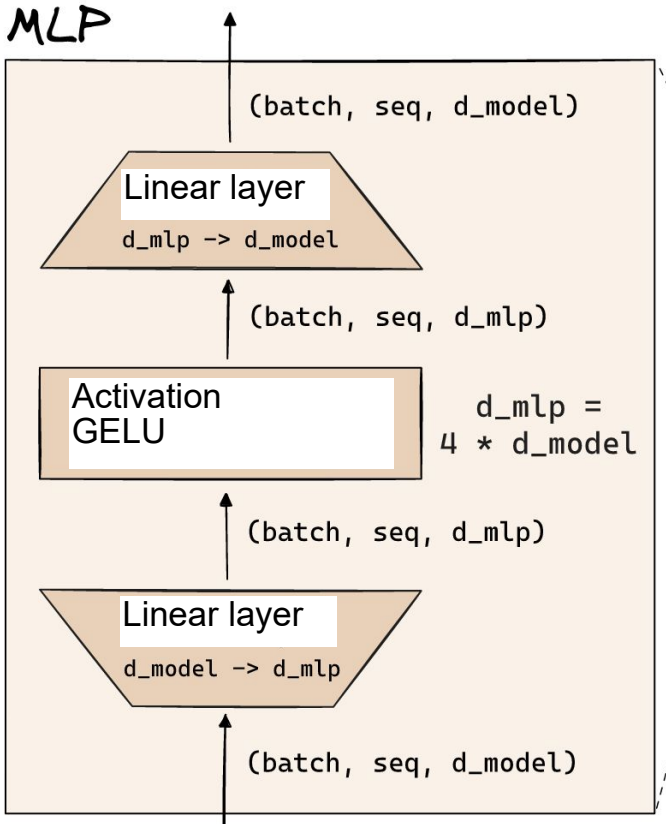
The values in both tables are learned during training.

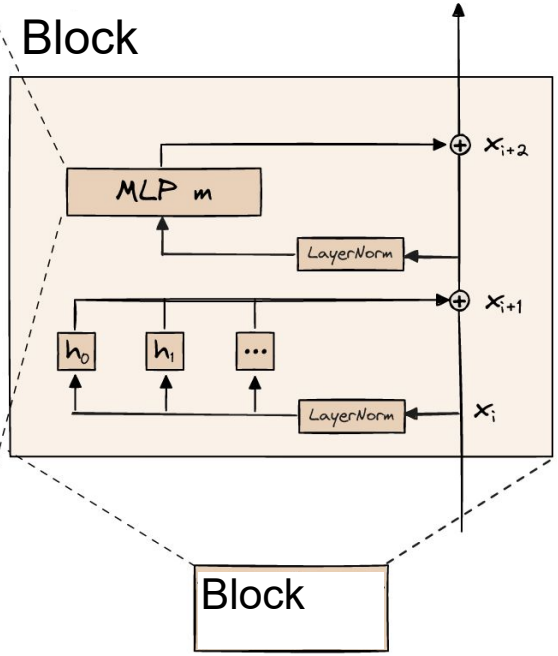
Add positional embeddings to token embeddings to represent both what is present and where it is.

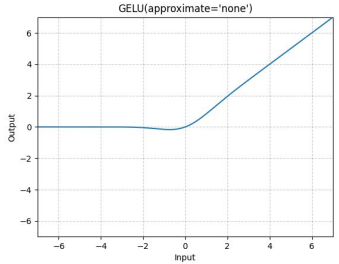
Transformer block



Attention gathers information from other positions; the MLP processes information at each position.







$$\mathrm{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-\frac{t^2}{2}}\, dt$$

GELU is a nonlinear transformation.
You do not need to memorize the integral.

Mix and expand the numbers, apply a nonlinear function, and return to the original vector length.

Several features can overlap in a vector

In N dimensions, at most N vectors can be mutually orthogonal.

Allowing slightly overlapping directions lets us fit many more.

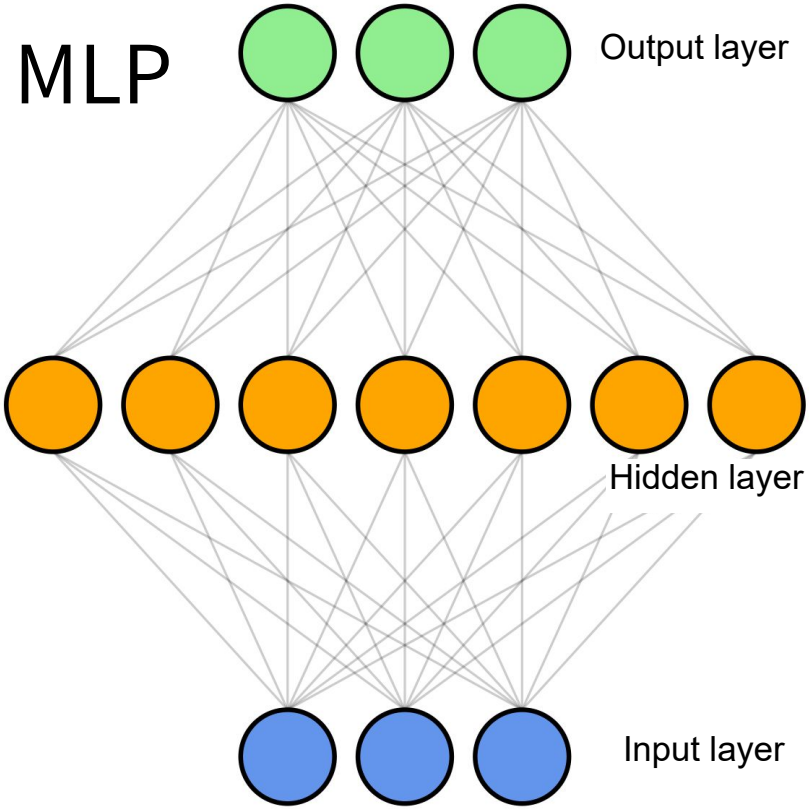
What to remember

A vector entry need not correspond to one concept.

More dimensions allow more representation, but do not guarantee better performance.

Sharing the vector space between features is called superposition.

Several features can share overlapping directions in one vector space. This slide is optional further study.



This MLP is a neural network with one hidden layer.

$$\mathrm{MLP}(x) = W_2(\sigma(W_1 x + b_1)) + b_2$$

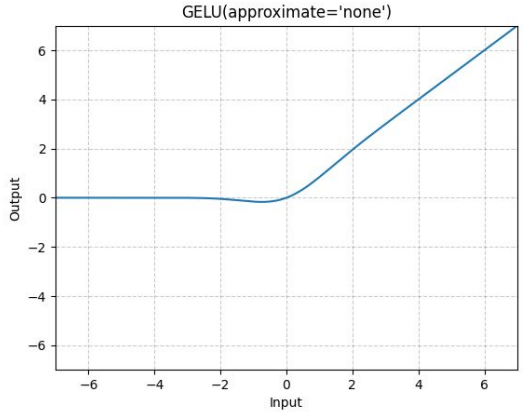
σ is a nonlinear activation function.

GPT2 uses a GELU-family function.

$$\mathrm{GELU}(x) = x\,\Phi(x) \approx \max(x, 0)$$

Φ(x) is the cumulative probability of the standard normal distribution (reference).

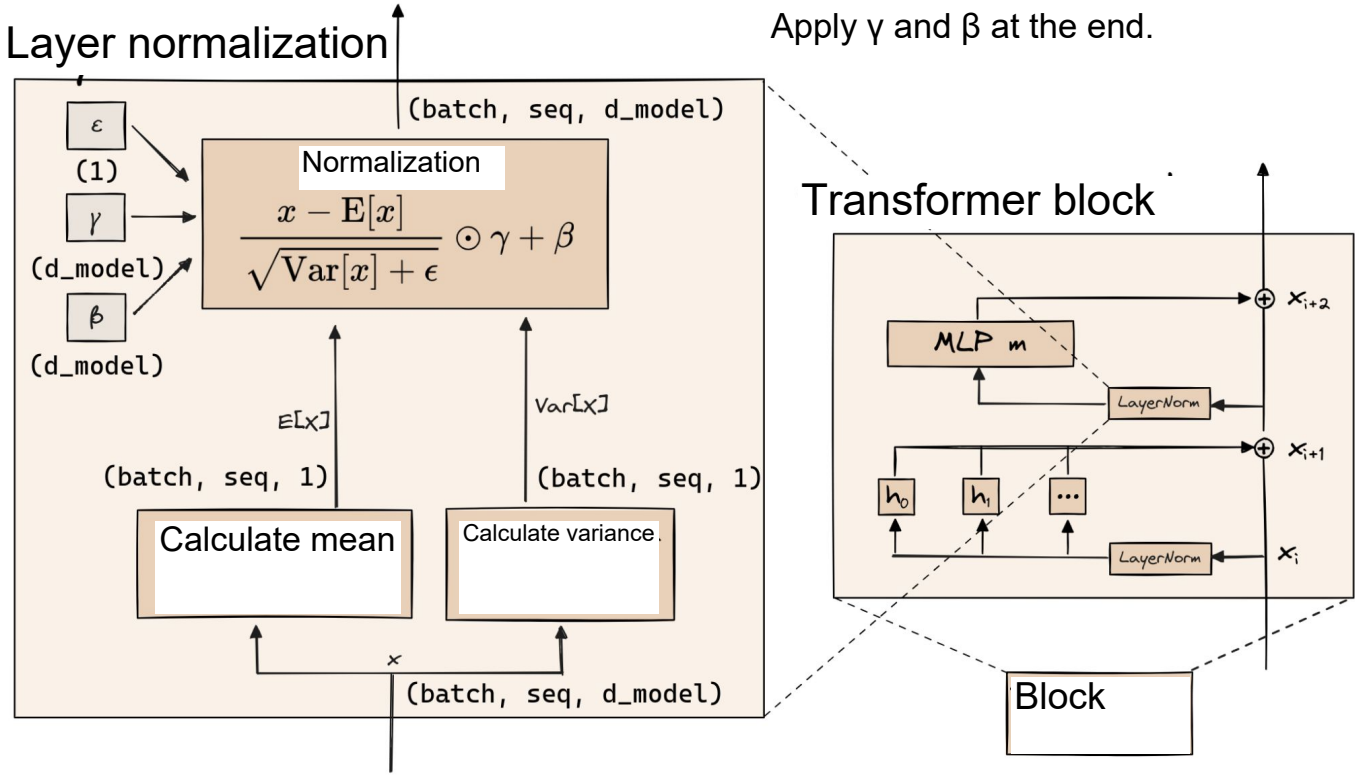
Other examples include ReLU and Tanh.



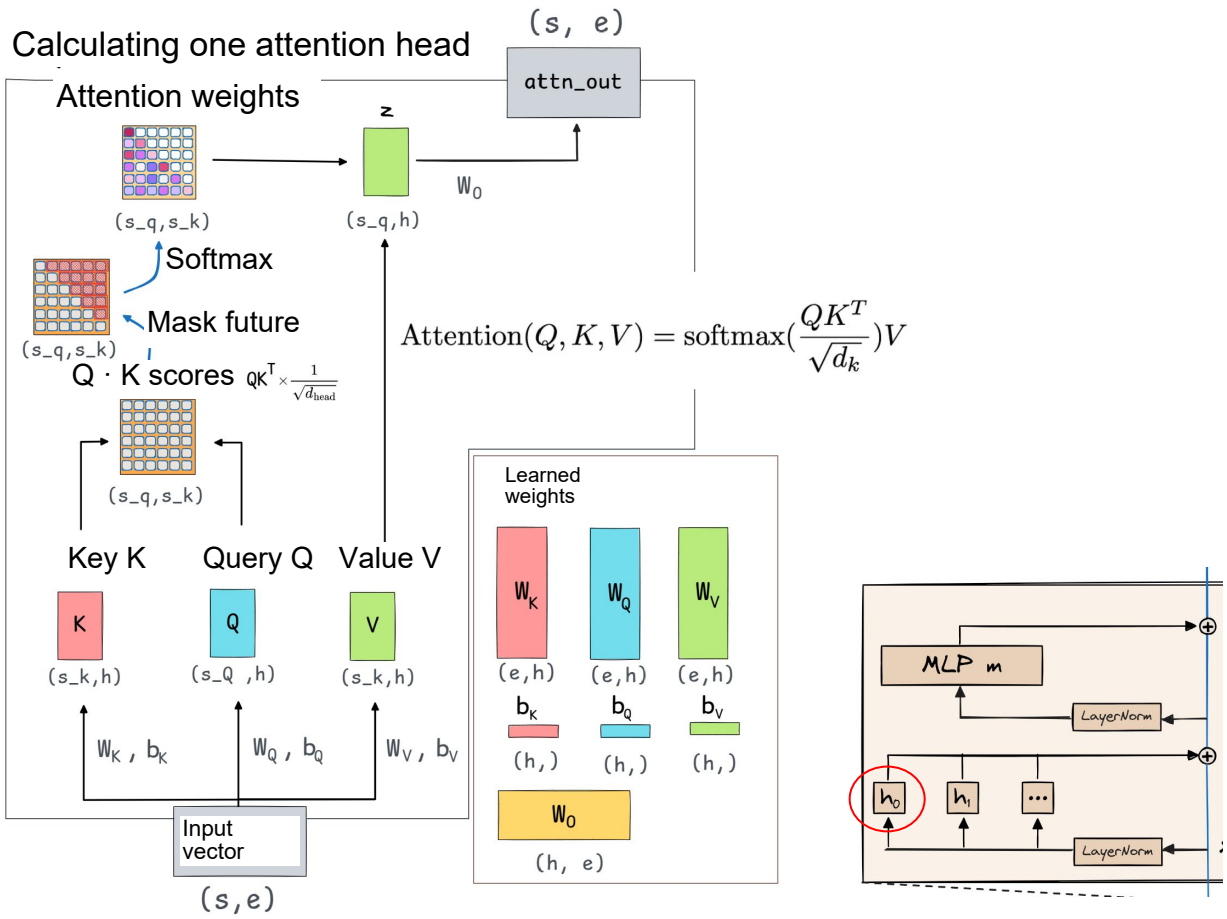
Process input information using connection weights together with a nonlinear transformation.

Adjust the numerical scale for each token.

Apply γ and β at the end.



Normalize one token's vector, then apply a learned scale and shift.



Use Q and K to determine attention weights, then mix V using those weights.

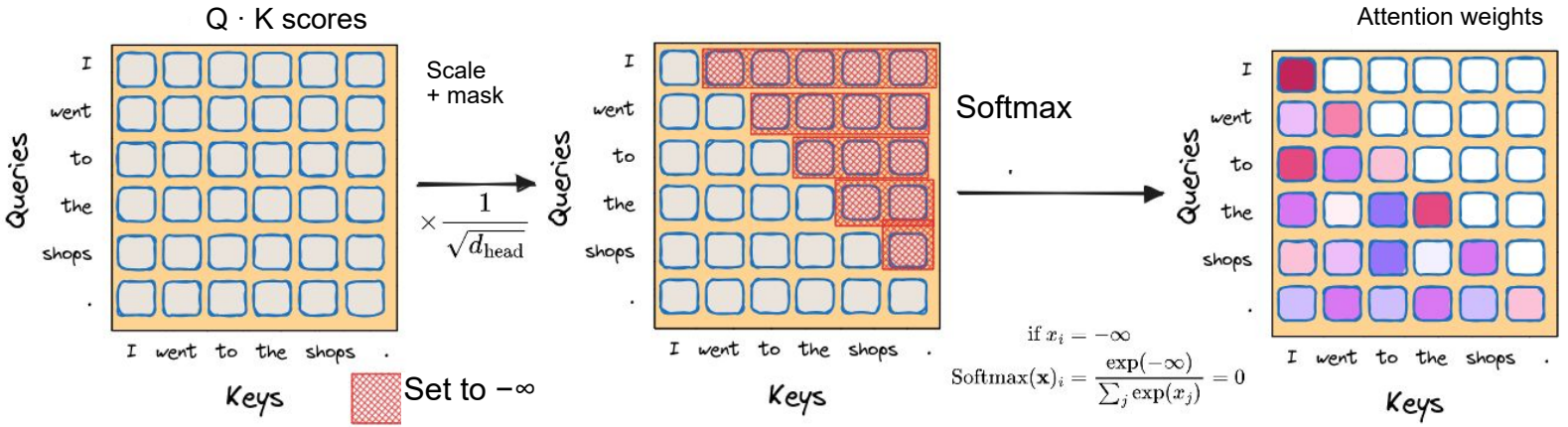
Causal masking: hiding the future

Block access to keys after the current position.

Prevent the model from reading answers that are not yet available.

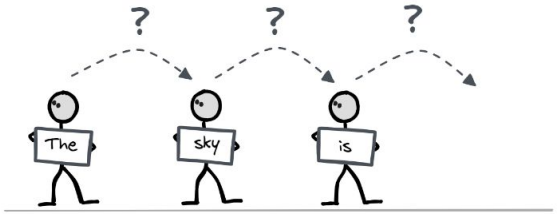
Block access to future positions.

Each row's weights sum to 1.



The current position can refer only to itself and earlier tokens.

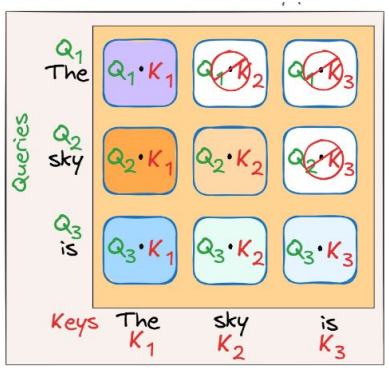
Each position uses earlier information to predict the next token.



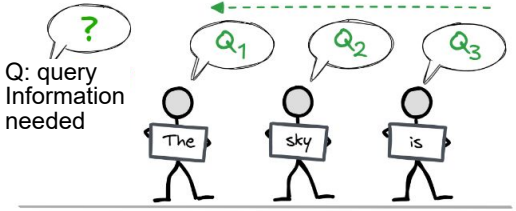
Mask future positions.
A position can see itself.



Rows: query positions Q
Columns: key positions K

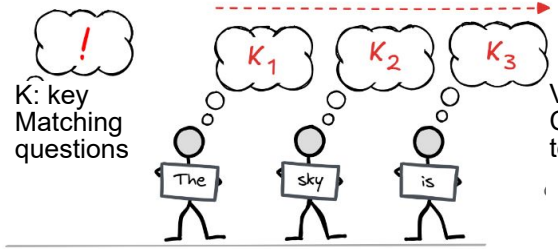


Q and K determine attention weights.
Use those weights to mix V.



Q: query
Information
needed

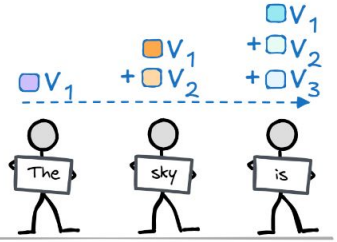
A position can also refer to itself.



K: key
Matching
questions

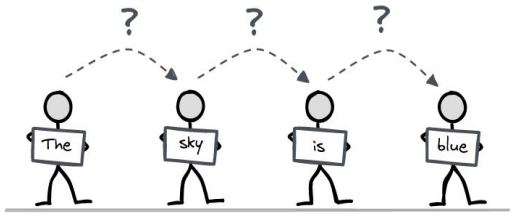


V: value
Content
to pass on



Where query Q matches key K, retrieve content V to pass on.

Refer to earlier tokens
to predict the next token.

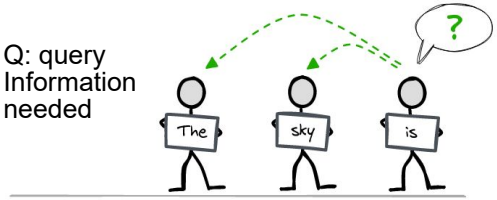


The MLP processes information
at each position. It does not
directly retrieve other positions.

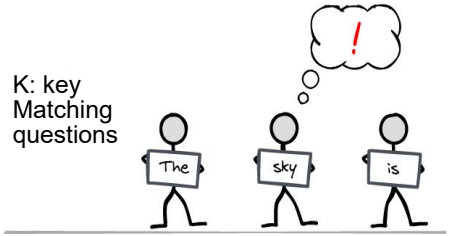


Attention gathers relevant
information from earlier positions.

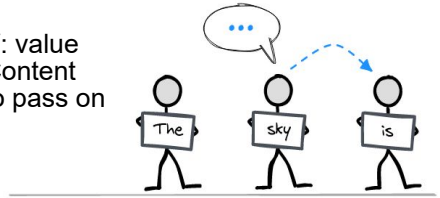
Q: query
Information
needed



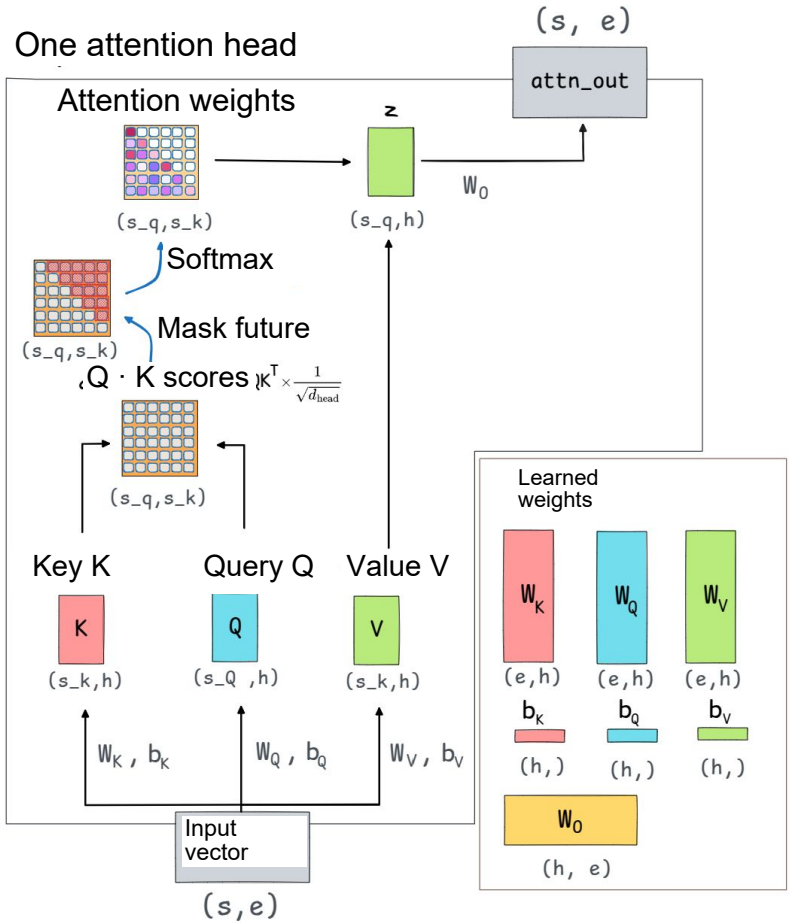
K: key
Matching
questions

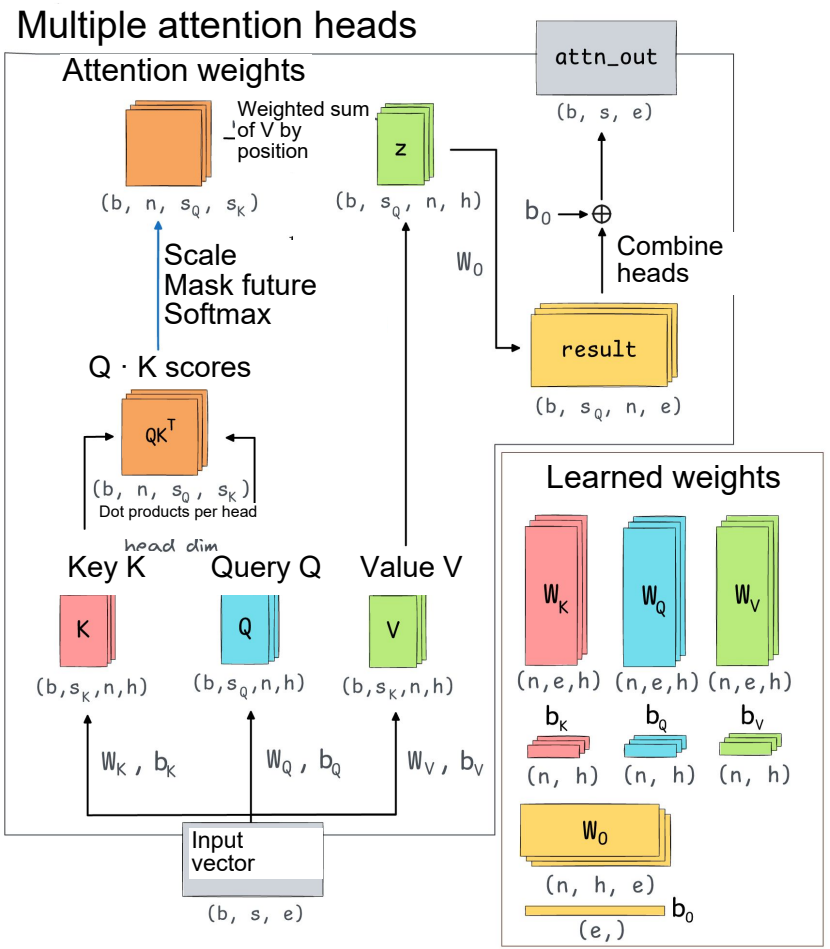


V: value
Content
to pass on



Attention gathers information from other positions; the MLP processes the information gathered.





Several heads gather information with their own attention weights, then combine the results.

Attention head

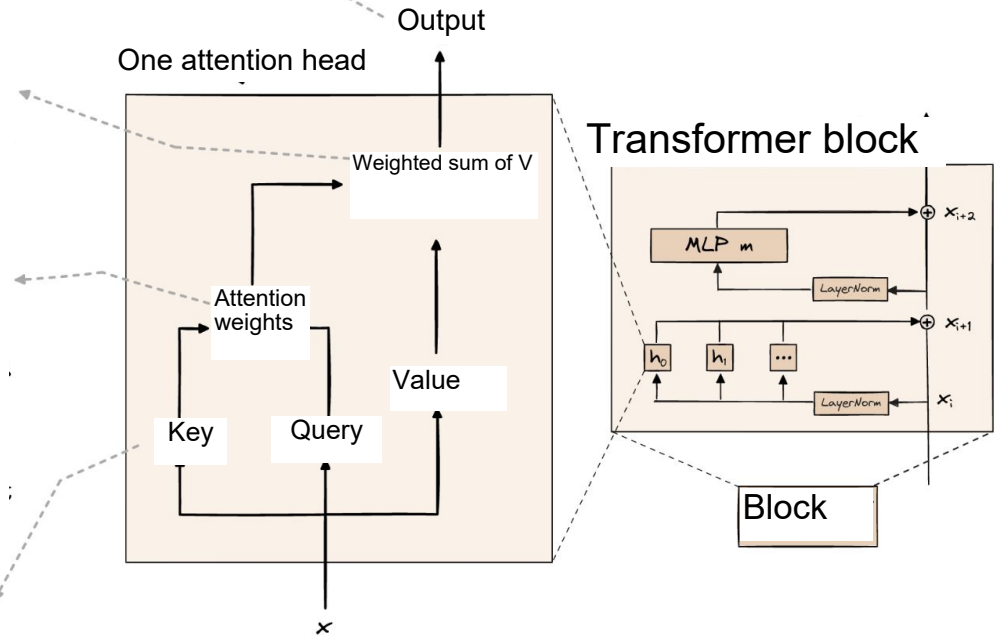
The output transformation sets the vector length for addition to the residual stream.

Take a weighted sum of V. A larger attention weight means a larger coefficient.

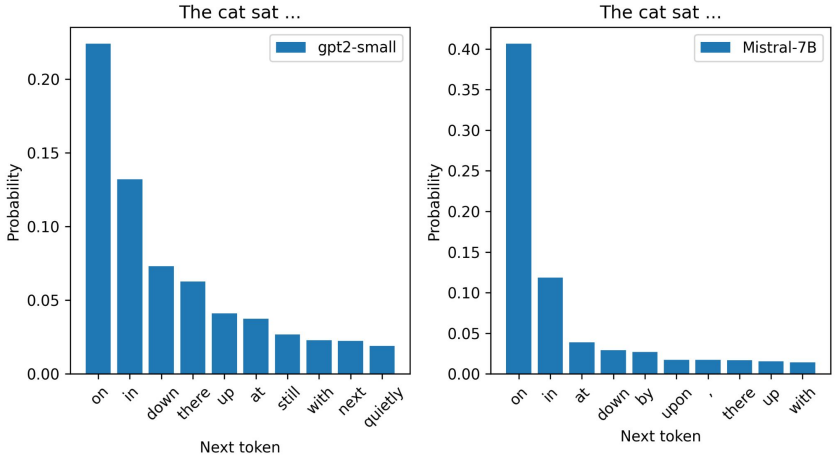
Weights describe which positions provide information and in what proportions.

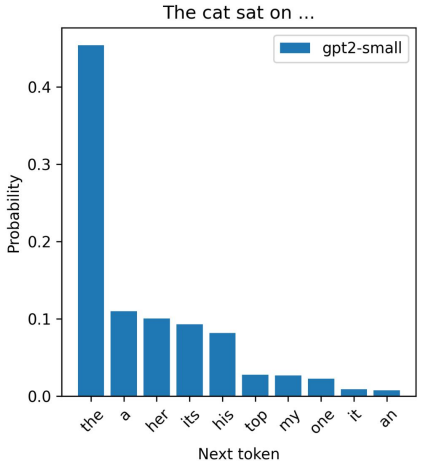
Set attention weights for future positions to zero.

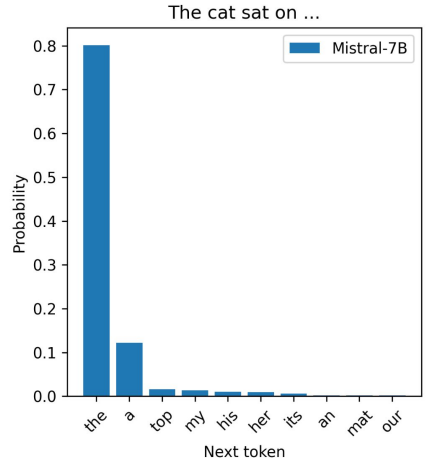
Create Q, K, V from the input using different transformations.

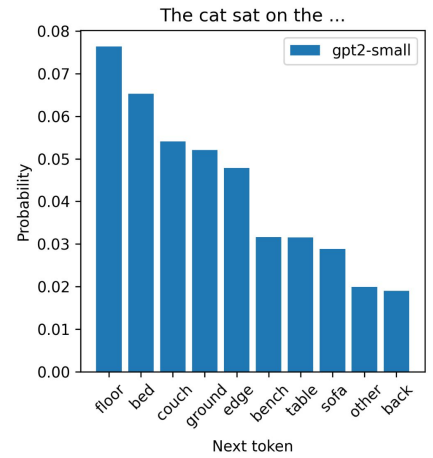


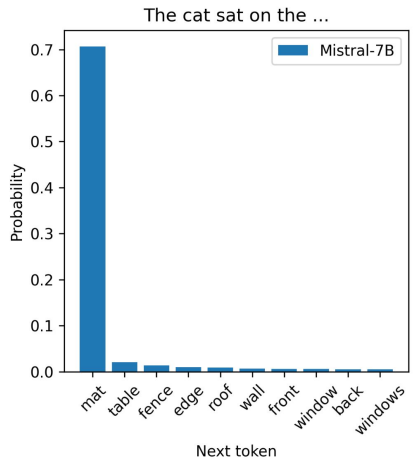
Create Q, K, V → scores → mask future → weighted sum of V → output transformation











Even for the same model, changing the preceding text changes next-token probabilities.

Axes
b: batch size · s: tokens · n: heads
e: model-vector length · h: head-vector length

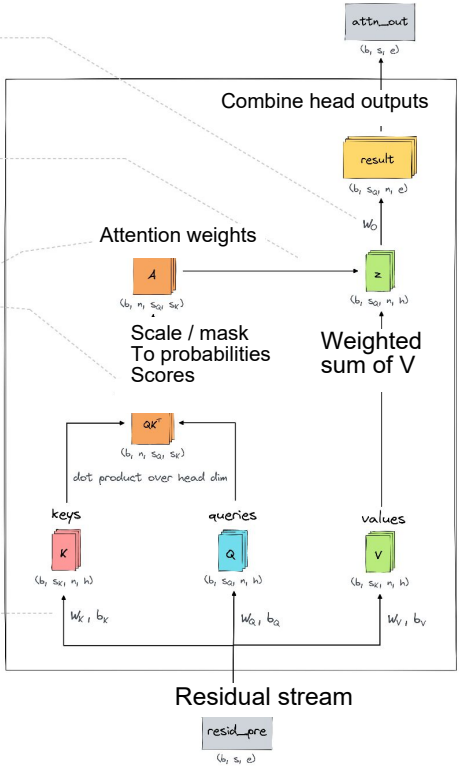
Apply the output transform
and combine head results.

Take a weighted sum of V
using the attention weights.

Take dot products of Q
and K
to calculate scores.

Mask the future,
then apply softmax.

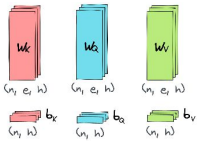
Create Q, K, V
separately
from the input.



Learned weights

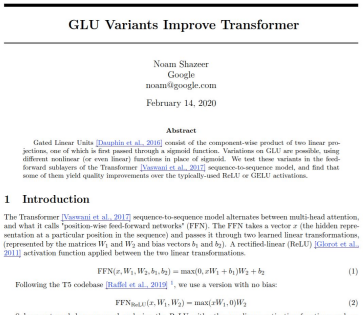


Learned weights



High-resolution source

Check the array shape at each arrow to follow an attention implementation.



GLU Variants Improve Transformer

Noam Shazeer
Google
noam@google.com

February 14, 2020

Abstract

Gated Linear Units [Dauphin et al., 2016] consist of the component-wise product of two linear projections, one of which is first passed through a sigmoid function. Variations on GLU are possible, using different nonlinear (or even linear) functions in place of sigmoid. We test these variants in the feed-forward sublayers of the Transformer [Vaswani et al., 2017] sequence-to-sequence model, and find that some of them yield quality improvements over the typically-used ReLU or GELU activations.

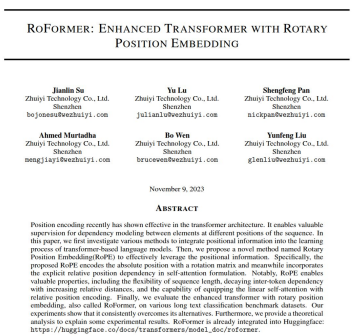
1 Introduction

The Transformer [Vaswani et al., 2017] sequence-to-sequence model alternates between multi-head attention, and what it calls "position-wise feed-forward networks" (FFN). The FFN takes a vector x (the hidden representation at a particular position in the sequence) and passes it through two learned linear transformations, (represented by the matrices W_1 and W_2 and bias vectors b_1 and b_2). A rectified-linear (ReLU) [Glorot et al., 2011] activation function applied between the two linear transformations.

$$\mathrm{FFN}(x, W_1, W_2, b_1, b_2) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (1)$$

Following the T5 codebase [Raffel et al., 2019] [1], we use a version with no bias:

$$\mathrm{FFN}_{\mathrm{ReLU}}(x, W_1, W_2) = \max(xW_1, 0)W_2 \quad (2)$$



ROFORMER: ENHANCED TRANSFORMER WITH ROTARY POSITION EMBEDDING

Jianlin Su, Zhuiyi Technology Co., Ltd., Shenzhen, bojonesu@wezhuiyi.com
Yu Lu, Zhuiyi Technology Co., Ltd., Shenzhen, julianlu@wezhuiyi.com
Shengfeng Pan, Zhuiyi Technology Co., Ltd., Shenzhen, nickpan@wezhuiyi.com
Ahmed Murtadha, Zhuiyi Technology Co., Ltd., Shenzhen, mengjiayi@wezhuiyi.com
Bo Wen, Zhuiyi Technology Co., Ltd., Shenzhen, brucewen@wezhuiyi.com
Yunfeng Liu, Zhuiyi Technology Co., Ltd., Shenzhen, glenliu@wezhuiyi.com

November 9, 2023

Abstract

Position encoding recently has shown effective in the transformer architecture. It enables valuable supervision for dependency modeling between elements at different positions of the sequence. In this paper, we first investigate various methods to integrate positional information into the learning process of transformer-based language models. Then, we propose a novel method named Rotary Position Embedding(RoPE) to effectively leverage the positional information. Specifically, the proposed RoPE encodes the absolute position with a rotation matrix and meanwhile incorporates the explicit relative position dependency in self-attention formulation. Notably, RoPE enables valuable properties, including the flexibility of sequence length, decaying inter-token dependency with increasing relative distances, and the capability of equipping the linear self-attention with relative position encoding. Finally, we evaluate the enhanced transformer with rotary position embedding, also called RoFormer, on various long text classification benchmark datasets. Our experiments show that it consistently overcomes its alternatives. Furthermore, we provide a theoretical analysis to explain some experimental results. RoFormer is already integrated into Huggingface: https://huggingface.co/docs/transformers/model_doc/roformer.

Keywords Pre-trained Language Models · Position Information Encoding · Pre-training · Natural Language Processing.

1 Introduction



SWISH: A SELF-GATED ACTIVATION FUNCTION

Prajit Ramachandran; Barret Zoph, Quoc V. Le
Google Brain
{prajit,barretzoph,qvl}@google.com

Abstract

The choice of activation functions in deep networks has a significant effect on the training dynamics and task performance. Currently, the most successful and widely-used activation function is the Rectified Linear Unit (ReLU). Although various alternatives to ReLU have been proposed, none have managed to replace it due to inconsistent gains. In this work, we propose a new activation function, named Swish, which is simply $f(x) = x \cdot \mathrm{sigmoid}(x)$. Our experiments show that Swish tends to work better than ReLU on deeper models across a number of challenging datasets. For example, simply replacing ReLUs with Swish units improves top-1 classification accuracy on ImageNet by 0.9% for Mobile NASNet-A and 0.6% for Inception-ResNet-v2. The simplicity of Swish and its similarity to ReLU make it easy for practitioners to replace ReLUs with Swish units in any neural network.

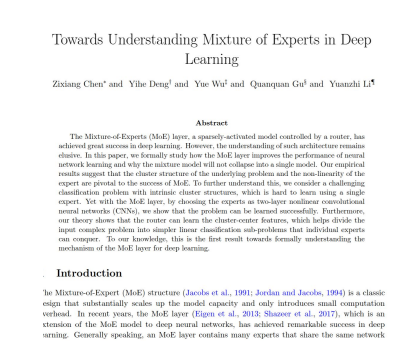
1 Introduction

At the heart of every deep network lies a linear transformation followed by an activation function $f(\cdot)$. The activation function plays a major role in the success of training deep neural networks. Currently, the most successful and widely-used activation function is the Rectified Linear Unit (ReLU) (Hahnloser et al., 2000; Jarrett et al., 2009; Nair & Hinton, 2010), defined as $f(x) = \max(x, 0)$. The use of ReLUs was a breakthrough that enabled the fully supervised training of state-of-the-art deep networks (Krizhevsky et al., 2012). Deep networks with ReLUs are more easily optimized than networks with *sigmoid* or *tanh* units, because gradient is able to flow when the input to the ReLU function is positive. Thanks to its simplicity and effectiveness, ReLU has become the default activation function used across the deep learning community.

While numerous activation functions have been proposed to replace ReLU (Maas et al., 2013; He et al., 2015; Clevert et al., 2015; Klambauer et al., 2017), none have managed to gain the widespread adoption that ReLU enjoys. Many practitioners have favored the simplicity and reliability of ReLU because the performance improvements of the other activation functions tend to be inconsistent

We propose a new activation function that we name *Swish*:

$$f(x) = x \cdot \sigma(x) \quad (1)$$



Towards Understanding Mixture of Experts in Deep Learning

Zixiang Chen* and Yihe Deng† and Yue Wu‡ and Quanquan Gu§ and Yuanzhi Li¶

Abstract

The Mixture-of-Experts (MoE) layer, a sparsely-activated model controlled by a router, has achieved great success in deep learning. However, the understanding of such architecture remains elusive. In this paper, we formally study how the MoE layer improves the performance of neural network learning and why the mixture model will not collapse into a single model. Our empirical results suggest that the cluster structure of the underlying problem and the non-linearity of the expert are pivotal to the success of MoE. To further understand this, we consider a challenging classification problem with intrinsic cluster structures, which is hard to learn using a single expert. Yet with the MoE layer, by choosing the experts as two-layer nonlinear convolutional neural networks (CNNs), we show that the problem can be learned successfully. Furthermore, our theory shows that the router can learn the cluster-center features, which helps divide the input complex problem into simpler linear classification sub-problems that individual experts can conquer. To our knowledge, this is the first result towards formally understanding the mechanism of the MoE layer for deep learning.

Introduction

he Mixture-of-Expert (MoE) structure (Jacobs et al., 1991; Jordan and Jacobs, 1994) is a classic esign that substantially scales up the model capacity and only introduces small computation verhead. In recent years, the MoE layer (Eigen et al., 2013; Shazeer et al., 2017), which is an xtension of the MoE model to deep neural networks, has achieved remarkable success in deep arning. Generally speaking, an MoE layer contains many experts that share the same network rchitecture and are trained by the same algorithm, with a gating (or routing) function that routes dividual inputs to a few experts among all the candidates. Through the sparse gating function, e router in the MoE layer can route each input to the top-K ($K \geq 2$) best experts (Shazeer et al., l al., 2021). This routing scheme only costs the enjoys fast inference time.

California, Los Angeles, CA 90095, USA; e-mail:
California, Los Angeles, CA 90095, USA; e-mail:
alifornia, Los Angeles, CA 90095, USA; e-mail:
California, Los Angeles, CA 90095, USA; e-mail:
lon University, Pittsburgh, PA, USA; e-mail:

arXiv:2002.05202v1 [cs.LG] 12 Feb 2020
arXiv:1710.05941v1 [cs.NE] 16 Oct 2017
arXiv:1706.03762v7 [cs.CL] 2 Aug 2023
104.09864v5 [cs.CL] 8 Nov 2023

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figur

Changing the architecture can change performance.
The reasons are not always fully understood.



Attentic

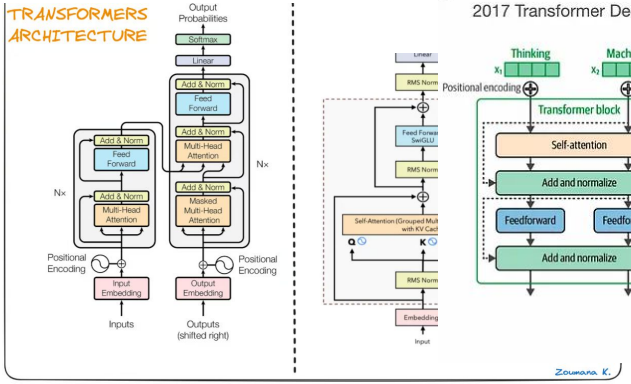
Ashish Vaswani* Google Brain avaswani@google.com
Noam [illegible] Google Brain noam@google.com
[illegible] Google Research nikip@google.com
[illegible] Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

* Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.
† Work performed while at Google Brain.
‡ Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.



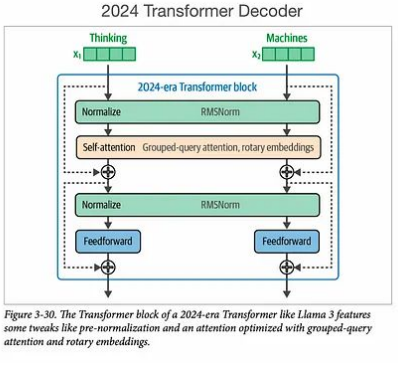


Figure 3-30. The Transformer block of a 2024-era Transformer like Llama 3 features some tweaks like pre-normalization and an attention optimized with grouped-query attention and rotary embeddings.

For today, understanding the basic GPT2 flow is enough. Architectural variations are the next step.